PCA / PRA — Plateforme de paiement RESILIENCE-ALPHA

> ***Statut : cible.*** *Ce document décrit ce que l'équipe vise et a décidé, pas ce qui est déjà codé. L'état réel du code est suivi dans* `constat/etat-implementation.md`*.*

> *Ticket : WOR-71 · Responsable : Cédric · Version 0.3 du 21/10/2026*
>
> ***Statut :*** *les actions automatiques décrites sont celles du code (moteur de remédiation* `services/remediation/`*, playbooks* `ansible/playbooks/remediation/`*), relevées dans* `constat/etat-implementation.md` *v0.3. Chaque fiche sera confirmée après les essais (§9). Chaque fiche se lit seule : l'assistant RAG peut la citer isolément.*

1. Objet et périmètre

- **PCA (plan de continuité)** : comment le service de paiement **continue** pendant l'incident, en mode normal ou dégradé.
- **PRA (plan de reprise)** : comment on **revient** à un état sain et vérifié après l'incident, avec quelles données.

Dans le périmètre	Hors périmètre
Plateforme Docker Compose du PC Hôte : front, authentification, API de paiement, base de données, WAF, supervision, détection ML, moteur de remédiation, assistant RAG	Infrastructure du campus, postes des autres groupes
Incidents de disponibilité, d'intégrité, défaillances internes, perte de l'hôte	Données réelles : toutes les données sont fictives

2. Rôles et responsabilités

2.1 Dans l'entreprise fictive

Le **moteur de remédiation** intervient en premier, dans son périmètre d'actions. L'**opérateur d'astreinte** reprend la main s'il échoue. Le **responsable de la production** décide des restaurations et des modes dégradés ; le **RSSI** qualifie les incidents de sécurité et décide de la préservation des preuves ; le **directeur de crise** arbitre entre reprise rapide et investigation. La **conformité** classe l'incident selon DORA et notifie l'ACPR, la **communication** informe les partenaires, le **DPO** évalue l'atteinte aux données personnelles.

2.2 Dans l'équipe du démonstrateur

Membre	Ce qu'il porte dans ce plan
Michael (Développement)	Moteur de remédiation, vérification de la reprise, cohérence des données
Reza (Infrastructure)	Playbooks Ansible, sauvegardes et restauration, remise à l'état sain
Léa (IA / Data)	Détection ML, assistant RAG
Axel (Cybersécurité)	WAF, SIEM, secrets, préservation des journaux
Cédric (Gouvernance)	Ce plan, classification des incidents, notifications, retour d'expérience

3. Niveaux de réponse et escalade

Les seuils viennent du coût d'un arrêt (`services-critiques.md`).

Niveau	Déclencheur	Qui agit	Gravité
N0 — Automatique	Alerte ML ou SIEM	Moteur de remédiation seul ; l'opérateur est informé	G1
N1 — Astreinte	Échec de la remédiation, **ou** service non rétabli à 5 min, **ou** incident d'intégrité ou de confidentialité (même si la remédiation a réussi)	Opérateur d'astreinte, RSSI informé	G2
N2 — Cellule de crise	Service non rétabli à 30 min, **ou** perte de données confirmée, **ou** compromission du moteur de remédiation	Directeur de crise, RSSI, conformité, communication	G3
N3 — Crise majeure	Service non rétabli à 2 h, **ou** perte du PC Hôte	Direction générale	G4

Règle de l'automate : l'automate fait **au plus 2 tentatives par incident** (action puis vérification). Après 2 échecs, il s'arrête, marque l'incident en échec et émet une escalade vers l'opérateur (N1, `docs/runbooks/reprise.md`). Une nouvelle alerte du même type pendant un incident en cours est fusionnée : seules les nouvelles sources sont bloquées. Il n'y a pas de limite d'un incident à l'autre. Après un redémarrage ou une restauration, le moteur **ignore toute nouvelle alerte pendant 180 s** (afflux de reconnexions des clients) ; il la journalise (`alert_suppressed`) sans agir.

4. Fiches scénarios

Fiche S1 — Déni de service sur l'API de paiement (attaque A)

- **Risque EBIOS :** R1 (hacktiviste, ER1) · gravité G4 si non traité
- **Signaux :** alerte ML de la famille `traffic` (débit, latence, erreurs, part de la première IP), après 3 fenêtres anormales consécutives
- **Actions automatiques :**
  1. **Blocage complet (403) au WAF** des sources qui pèsent au moins 20 % du trafic suspect ; la liste est rechargée à chaud
  2. Transaction de test ; si elle échoue, redémarrage de l'API
  3. Vérification : transaction de test réussie à travers le WAF
- **Données :** aucune restauration. Mais l'API du socle garde ses données en mémoire : le redémarrage (action 2) efface les transactions
- **Objectif :** RTO ≤ 5 min (cible démonstrateur ≤ 60 s) · RPO = 0 (pas tenable tant que l'API est en mémoire)
- **Escalade :** N1 après 2 tentatives en échec ; une nouvelle alerte pendant l'incident ne bloque que les nouvelles sources
- **Retour à la normale : pas de levée automatique** des blocages. L'opérateur lève la liste (`waf-unblock-all.yml`) ; la remise à l'état sain le fait aussi
- **Risques résiduels :**
  - L'attaquant change d'adresse : le blocage par source ne suffit plus (un vrai déni de service distribué exige une protection en amont, hors de portée du prototype)
  - Un partenaire légitime au fort volume peut être **bloqué** à tort, jusqu'à une action manuelle

Fiche S2 — Injection applicative et altération de données (attaque B)

- **Risque EBIOS :** R2 (cybercriminel, ER2) · gravité G4
- **Signaux :** alerte ML de la famille `injection` (motifs d'injection et blocages du WAF dans les journaux), après 3 fenêtres anormales consécutives. Une seule requête d'injection bloquée par le WAF suffit à la déclencher

- **Actions automatiques :**
  1. Blocage complet (403) de la source au WAF
  2. Constat d'intégrité : appel à `/health` (qui répond toujours « ok » : ce n'est pas un vrai contrôle)
  3. **Restauration à l'état sain connu** : recréation du conteneur de l'API. L'API étant en mémoire, elle repart des données de départ ; **toutes les transactions, légitimes comprises, sont perdues**
  4. Vérification : transaction de test réussie à travers le WAF
- **Pas automatisé :** copie des preuves (`scripts/export-logs.sh`, à lancer à la main **avant** la restauration si possible), rotation des secrets, validation humaine
- **Objectif :** RTO ≤ 5 min (cible ≤ 3 min) · RPO = 0 visé, **non tenu** par la restauration actuelle
- **Escalade :** automatique seulement en cas d'échec. Procédure : l'opérateur informe le RSSI de tout incident d'intégrité, même résolu ; la conformité évalue s'il est majeur au sens de DORA (une perte de données après un accès malveillant suffit)
- **Retour à la normale :** après correction de la vulnérabilité exploitée. D'ici là, la règle WAF reste active
- **Risques résiduels :**
  - La vulnérabilité est toujours dans le code : le WAF n'est qu'un pansement
  - Une porte dérobée a pu être déposée ; restaurer les données ne la supprime pas
  - Une alerte à tort efface quand même toutes les transactions (voir S4)
  - Une tentative d'injection que le WAF a bloquée déclenche quand même la restauration, donc la perte des transactions
  - Pendant les 180 s qui suivent la restauration, une nouvelle attaque n'est pas traitée automatiquement : l'opérateur surveille le tableau de bord
  - Sans export manuel, les preuves de l'attaque restent dans Loki, mais rien ne les protège d'une remise à l'état sain

Fiche S3 — Panne d'un service sans attaque

- **Couvre :** arrêt brutal d'un conteneur, saturation mémoire, base injoignable (testé par le chaos engineering, WOR-76), service de reconnaissance faciale indisponible (l'administrateur enrôlé ne peut plus se connecter au back-office)
- **Signaux :** contrôle de santé en échec, métriques absentes
- **Actions :** Docker Compose relance le conteneur (`healthcheck`) ; sinon, l'opérateur applique le runbook
- **Objectif :** RTO ≤ 5 min · RPO = 0
- **Escalade :** pas d'action automatique : le moteur ne réagit qu'aux alertes ML. L'opérateur relance le service (`docs/runbooks/reprise.md`)
- **Risques résiduels :** une cause profonde (disque plein, fuite mémoire) fera revenir la panne ; aucune connexion de secours au back-office n'est prévue si la reconnaissance faciale reste indisponible

Fiche S4 — Remédiation déclenchée à tort (faux positif du modèle ML)

- **Risque :** non intentionnel, cité dans `ebios-rm.md`
- **Signaux :** incident ouvert sans attaque en cours, sources bloquées qui sont des clients légitimes
- **Déjà survenu :** le 21/10 entre 12 h 36 et 13 h 34, sur du trafic légitime, 2 fausses alertes d'injection ont chacune déclenché une restauration (`ml/training/README.md`). Le détecteur a été réglé ensuite (3 fenêtres, seuil plus strict) : ce scénario reste possible, mais moins fréquent

- **Ce que fait le code :** une seule alerte ML suffit à déclencher toutes les actions, y compris la restauration. Il n'y a pas de second signal exigé (l'alerte combine déjà deux méthodes de détection, mais sur les mêmes données)
- **Conduite à tenir :** l'opérateur lève les blocages à tort (`waf-unblock-all.yml`) et note l'incident pour le réentraînement du modèle
- **Risques résiduels :** coupure provoquée par la défense elle-même ; après une fausse alerte d'injection, **perte de toutes les transactions**. La règle des deux signaux reste recommandée pour la production (`recommandations.md` REC-3)

Fiche S5 — Moteur de remédiation défaillant ou détourné

- **Risque EBIOS :** R3 (initié malveillant, ER6) · gravité G4
- **Signaux :** action hors du périmètre autorisé, appel au moteur depuis un composant non autorisé, rafale d'actions
- **Conduite à tenir :**
  1. Arrêter le moteur de remédiation
  2. Passer en **surveillance manuelle** : l'opérateur suit le tableau de bord et applique les fiches à la main
  3. Préserver les journaux du moteur
- **Escalade :** N2 immédiat
- **Prévention (en place) :** alertes signées HMAC, refus des alertes trop anciennes ou rejouées, liste fermée de playbooks, IP de l'hôte jamais bloquées, écoute locale, journal des événements. **Limite :** le moteur tourne sur l'hôte avec les droits Docker de l'utilisateur
- **Risques résiduels :** pendant l'arrêt du moteur, plus aucune reprise automatique ; on retombe sur un RTO manuel de 30 min ou plus

Fiche S6 — Suspicion de fuite de données

- **Risque EBIOS :** R5 (ER3) · gravité G3
- **Signaux :** volume de lecture anormal, requêtes inhabituelles sur la base, sortie de données vers une destination inconnue
- **Conduite à tenir :** couper l'accès de la source, préserver les preuves, **ne pas restaurer** (restaurer n'efface pas une fuite), prévenir le DPO
- **Escalade :** N2
- **Remarque :** non joué pendant la démo ; la réponse diffère d'une simple remise en service

Fiche S7 — Perte du PC Hôte (PRA)

- **Risque EBIOS :** R7 (rançongiciel) ou panne matérielle · gravité G4
- **Conduite à tenir :**
  1. Préparer une machine de remplacement (prérequis du README, WOR-49)
  2. Cloner le dépôt Git, redéployer avec Docker Compose et les playbooks Ansible
  3. Régénérer les secrets : Vault tourne sur le PC Hôte, son contenu est perdu avec lui. Le déploiement en crée de nouveaux quand `infra/.env` est absent
  4. Restaurer la dernière sauvegarde de la base, **si elle existe hors de l'hôte**
- **Objectif :** aucun objectif chiffré tenable avec un seul hôte
- **Risques résiduels : perte de toutes les données depuis la dernière copie hors hôte**. C'est le principal écart du démonstrateur (voir §8)

5. Mode dégradé

Quand la reprise automatique échoue, on maintient ce qui peut l'être :

Service	En mode dégradé	Interdit
Paiement	Limitation de débit stricte : on accepte moins de transactions plutôt qu'aucune	—
WAF	Mode de filtrage strict	**Le contourner** pour rétablir le trafic plus vite
Consultation	Lecture seule, voire suspendue pour soulager la base	—
Supervision	Surveillance manuelle sur le tableau de bord	—
Assistant RAG	Peut être arrêté pour libérer des ressources	Lui donner des droits d'action (WOR-57)

6. Choix de restauration

Élément	Où se trouve la référence saine	Comment on restaure	Fréquence	Responsable
Code et configuration des services	Dépôt Git, version taguée (WOR-80)	Redéploiement Docker Compose et Ansible	À chaque livraison	Reza
Règles du WAF	Dépôt Git	Liste d'IP bloquées : rechargement à chaud ; autres règles : redémarrage du WAF	À chaque modification	Axel
Secrets	HashiCorp Vault, sur le PC Hôte	Rotation puis redémarrage des services concernés	Après tout incident d'intégrité	Axel
Données de l'API	**En mémoire dans le socle, aucune sauvegarde**	Recréation du conteneur : retour aux données de départ (`restore-service.yml`)	— (minimum souhaité : base PostgreSQL et `pg_dump` avant chaque essai)	Reza, Michael
Journaux	Loki, rétention 7 jours, sur le PC Hôte	**Jamais écrasés** ; copiés avant toute restauration avec `scripts/export-logs.sh` (à la main pour l'instant)	En continu	Axel
Modèle ML	Dépôt Git (`ml/`)	Rechargement de la version validée	À chaque entraînement	Léa
Index du RAG	Documents du dépôt	Réindexation	À chaque mise à jour des documents	Léa

> ***Ne pas confondre restauration et remise à l'état sain.*** *La remise à l'état sain (`scripts/reset-known-good.sh`, WOR-90) supprime les volumes pour rejouer un essai : elle efface les transactions,* ***mais aussi les journaux Loki et les métriques Prometheus****, donc les preuves. Elle ne doit* ***jamais*** *servir à reprendre après un incident réel.*

7. Communication et notification

Destinataire	Quand	Quoi	Qui
Opérateur d'astreinte	Dès N0	Alerte et état de la reprise sur le tableau de bord	Automate

Destinataire	Quand	Quoi	Qui
Partenaires commerçants	Dès N1 si leurs transactions échouent	Nature de la gêne, heure de reprise estimée	Communication
ACPR, incident majeur au sens de DORA	Notification initiale dans les 4 h après la classification en majeur (et au plus tard 24 h après la détection), rapport intermédiaire sous 72 h après la notification initiale, rapport final sous 1 mois après le dernier rapport intermédiaire	Formulaire réglementaire, à partir des journaux du SIEM	Conformité
CNIL	Sous 72 h en cas de violation de données personnelles	Notification RGPD	DPO

Délais DORA : règlement délégué (UE) 2025/301. L'assistant RAG peut préparer un brouillon de notification à partir des journaux ; un humain le valide et l'envoie.

8. Risques résiduels après la reprise

Risque résiduel	Pourquoi il subsiste	Traitement proposé
L'attaquant revient, depuis une autre source	Le blocage est fait par source, sur un seul hôte	Protection anti-déni de service en amont (contexte industriel)
Vulnérabilité toujours présente après une injection	Le WAF bloque, mais le code n'est pas corrigé	Correctif et test de non-régression avant de lever le niveau N1
Porte dérobée ou altération ancienne	Restaurer les données ne nettoie pas tout	Analyse après incident à partir des journaux conservés
Hôte unique, sans sauvegarde hors site	Choix du prototype	Voir les limites industrielles, `recommandations.md` §3
Boucle de remédiation	Un incident persistant peut faire redémarrer un service en boucle	2 tentatives par incident, puis escalade ; pas encore de limite entre incidents
Faux positifs ML	Modèle entraîné sur du trafic normal seulement ; une alerte suffit à restaurer	Évaluer le modèle sur des attaques ; en production, deux signaux pour les actions destructrices (REC-3)
Droits élevés du moteur de remédiation	Nécessaires pour agir sur les conteneurs	Périmètre limité, authentification, journalisation (WOR-64)

9. Tests et maintien du plan

Test	Ticket	Scénarios vérifiés	Résultat
Premiers tests de restauration	WOR-75	S2, S7 (en partie)	À remplir
Chaos engineering	WOR-76	S3, S4	À remplir
Démo complète répétée 3 fois	WOR-78	S1, S2	À remplir, temps dans `rto-rpo.md` §6

Le plan est mis à jour après chaque test et après chaque incident (retour d'expérience, DORA art. 13).

10. Décisions en attente

1. Restauration après une injection : garder la recréation automatique (perte des transactions) ou ajouter une validation humaine et une vraie restauration de base : équipe, Cédric.
2. Brancher l'API du socle sur PostgreSQL et sauvegarder avant chaque essai : Reza, Michael.

3. Levée automatique des blocages, limite entre incidents, export des preuves avant restauration : Michael.
4. Ne pas restaurer quand le WAF a bloqué toutes les requêtes d'injection : Michael, Léa.
5. Connexion de secours au back-office si la reconnaissance faciale est indisponible : Axel, Michael.